CASE STUDY 1

CodeQL

Find the bug class, not just the bug

THE PROBLEM

The same vulnerability wears many disguises. Grep finds the string you thought of; attackers use the one you didn't. Real vulnerabilities are a data-flow problem — untrusted input reaching a dangerous sink — which pattern-matching can't follow across a program.

HOW CODEQL SOLVES IT

Code as data

Compiles source into a queryable database of structure + data flow; QL describes the shape of a bug and finds every instance.

Models as data (MaD)

One sink-model row teaches the whole query suite a new library's sinks — no new query, picked up by code scanning automatically.

One engine, two surfaces

Scans application code AND GitHub Actions workflows (code-injection, untrusted-checkout), with fixes in the PR via Copilot Autofix.

LIVE DEMO On a 60-line app, the default suite finds XSS + path traversal but **misses the node:sqlite SQL injection**. One sink-model row makes it find both variants — and stay silent on the safe parameterized query. A 25-line QL query shows the source→sink taint path.

TAKEAWAYS

- Code is data — turns “is this safe?” into a precise, repeatable query.
- Bug class > bug — catch every instance, not the one you grepped for.
- Modeling is the lever — extend the engine's model, rarely write a new query.

CASE STUDY 2

Breaking the chain

Disrupting supply chain attacks on npm & GitHub Actions

THE PROBLEM

Modern supply-chain attacks self-replicate: compromise a maintainer → steal credentials (install scripts, hijacked CI) → propagate into every package those tokens can publish → repeat. One phished maintainer becomes malware in hundreds of packages within hours.

BREAK EVERY LINK (2026 DEFENSES)

Compromise

npm 72-hr read-only on account takeover · actions/checkout refuses untrusted fork code (loud allow-unsafe-pr-checkout flag) · workflow trigger policies · read-only cache.

Steal credentials

npm 12: install scripts OFF by default · trusted publishing (OIDC) removes the long-lived NPM_TOKEN entirely · Actions network firewall.

Propagate

Staged publishing + 2FA gate (npm stage publish) · Dependabot cooldown (security updates never delayed) · instant credential revocation.

LIVE DEMO Demo 1: the same `npm install` runs a malicious postinstall on npm 11; **npm 12 blocks it by default**. Demo 2: an issue title (`...; echo PWNED; #`) executes code in a workflow and leaks a secret — then an env-var fix neutralizes it (CodeQL flags it critical).

TAKEAWAYS

- No single fix — layers; break any link and the self-replicating loop stalls.
- Secure-by-default wins — biggest gains protect people who change nothing.
- Kill the long-lived token — OIDC + staged publishing remove what attacks steal.